MODULE 01 · EXAM STRATEGY - ALL DOMAINS

The Developer's Mental Model

Eight ideas that make the rest of this course make sense – and that most exam questions quietly assume.

TYPE Foundations MODULE 1 of 34 SCENARIOS 2

admin@sthithapragna.com

You are not renting machines. You are **calling services**, and the services are already running.

The eight ideas

IDEA	WHAT IT MEANS FOR YOU
Everything is an API	Anything the console does, your code can do
Identity is the perimeter	Access is decided by IAM, not by network location
Managed means operated, not free	AWS runs it, and you still configure and pay for it
Compute is stateless	Anything you need later must be written somewhere
Everything fails	Design for retry, and therefore for idempotency
Consistency has a price	Strong costs more and is not always needed
Limits exist everywhere	Every service throttles, and quotas are per account and Region
Loose coupling is the default	Components communicate through queues, topics and events

READ THIS TWICE

Worth screenshotting now and revisiting at the end. Every module in this course is an application of one or more of these.

One: everything is an API

- Every AWS service is an **HTTPS API**. The console is a client of it, like your code.
- So anything you can do by clicking, you can do from a script or a template.
- **CloudTrail records those API calls**, which is why it can tell you who changed what.
- It also means infrastructure can be code, because creating a resource is just a call.
- The exam almost never asks about the console. It asks what your code or template does.

> ON THE EXAM
>
> This is why 'do it in the console' is rarely the intended answer. The console is for looking; the API is for building.

Two: identity is the perimeter

- There is no inside. A request from your own Lambda function is authenticated like any other.
- **Every API call is signed**, and every call is authorised against IAM policy.
- So 'it is in our VPC' grants nothing. A role and a policy do.
- Roles give **temporary credentials**, which is why long-lived keys are almost always wrong.
- Security is 26% of this exam, and nearly all of it flows from this idea.

> ON THE EXAM
>
> The habit worth building now: when something cannot access something else, ask which identity is making the call before you look at anything else.

Three: managed means operated

- AWS handles the **undifferentiated** work: patching, replacement, replication, scaling machinery.
- You still choose the **configuration**, and the configuration is what the exam tests.
- AWS runs DynamoDB. You choose the partition key, and a bad one still throttles.
- AWS runs Lambda. You choose the memory, timeout and concurrency.
- **Managed does not mean you cannot misuse it.** Most exam scenarios are misconfiguration.

> ON THE EXAM
>
> This is the shape of nearly every question: the service is working correctly, and the configuration is wrong. Look at the settings before the service.

Four: compute is stateless

- A Lambda execution environment, a container or an instance can disappear at any moment.
- **Anything you need later must be written somewhere durable.** Memory and local disk are not.
- Sessions go to DynamoDB or ElastiCache, not into process memory.
- Uploads go to S3, not to a local directory.
- Reuse across invocations is an **optimisation** you may benefit from, never a guarantee.

> ON THE EXAM
>
> The subtle version: Lambda often reuses an environment, so cached state usually survives. Code that depends on that works in testing and fails unpredictably later.

Five: everything fails

- Networks drop, services throttle, and instances are replaced. This is normal operation.
- AWS SDKs **retry automatically**, so your code will be called more than once eventually.
- **That makes idempotency a requirement, not a refinement.**
- Queues deliver at least once. Events can arrive twice. Timeouts leave you genuinely unsure.
- Design so that processing the same message twice is harmless.

> ON THE EXAM
>
> The timeout is the case that forces this: the operation may have succeeded and the response been lost, so a retry without protection charges the customer twice.

Six: consistency has a price

- **Durability** is whether data survives. **Availability** is whether you can reach it now.
- **Strongly consistent** reads always see the latest write, and cost more.
- **Eventually consistent** reads may briefly lag, and cost half as much in DynamoDB.
- Most reads tolerate a lag of milliseconds. A balance check does not.
- Distributed replicas are why the choice exists. It is not an implementation detail.

> ON THE EXAM
>
> The habit: ask whether this particular read would be harmed by data a few milliseconds old. Usually not, and the default is the cheaper one for a reason.

Seven: limits exist everywhere

- Every service has quotas, and every service throttles when you exceed them.
- Most quotas are **per account and per Region**, so workloads share them.
- One noisy workload can therefore starve another in the same account.
- Some quotas are **adjustable** by request. Some are hard and cannot be raised.
- Hard numbers decide exam questions: 15 minutes, 29 seconds, 400 KB, 5 GB.

> ON THE EXAM
>
> Knowing roughly thirty numbers is worth more than knowing a hundred features, because a stated size or duration often eliminates options outright.

Eight: loose coupling is the default

- Components communicate through **queues, topics and events** rather than direct calls.
- A queue lets a slow consumer fall behind without the producer failing.
- A topic lets you add a consumer without changing the producer.
- That is why the integration services carry so much weight on this exam.
- Direct synchronous calls between components are usually the wrong answer.

> ON THE EXAM
>
> When two options both work, the more loosely coupled one is usually intended. That heuristic is genuinely predictive on this exam.

Regions and Availability Zones

- A **Region** is a geographic area. Resources and most quotas are Regional.
- An **Availability Zone** is one or more isolated data centres within a Region.
- Managed services such as S3, DynamoDB and Lambda span AZs for you automatically.
- Data does **not** move between Regions unless you configure it to.
- A few services are global: **IAM, CloudFront and Route 53**, with endpoints in us-east-1.

> ON THE EXAM
>
> For a developer the practical point is that Region is part of a resource's identity. The same bucket name in another Region is a different bucket.

The account as a boundary

- An AWS account is the hardest boundary AWS offers, for security, quotas and billing.
- Separate accounts per environment is the common pattern, and prevents dev affecting prod.
- Nothing crosses an account boundary **by default**. Every crossing is explicit.
- Cross-account access needs **both** sides to agree: a role or resource policy, and an identity policy.
- Within one account, either side granting access is enough.

> ON THE EXAM
>
> That last pair is worth remembering from the start: same account is OR, cross-account is AND. It resolves a surprising number of questions.

The layers of an AWS application

LAYER	WHAT YOU WILL MEET IN THIS COURSE
Compute	Lambda, ECS and Fargate, Elastic Beanstalk
Storage and data	S3, DynamoDB, RDS and Aurora, ElastiCache
Integration	API Gateway, SQS, SNS, EventBridge, Step Functions
Security	IAM, Cognito, KMS, Secrets Manager
Deployment	CloudFormation, SAM, CDK, CodePipeline, CodeDeploy
Observability	CloudWatch, X-Ray, Application Signals

ON THE EXAM

Six layers, and the exam draws from all six. Recognising which layer a question is about is often the fastest way to eliminate options.

The cost model

- **Pay per use** charges for what you consume, and nothing while idle.
- **Provisioned** charges for capacity whether or not you use it, and is cheaper when steady.
- The exam asks for **relative** cost, never dollar figures.
- Predictability decides between them, not volume.
- In serverless you pay for **time**, so faster is frequently cheaper.

> ON THE EXAM
>
> That last line is genuinely counterintuitive and worth carrying: giving a Lambda function more memory often reduces the bill, because the duration falls further than the rate rises.

Request-response or event-driven?

REQUEST-RESPONSE

The caller waits

- A client needs an answer now
- API Gateway in front of Lambda
- Latency is user-visible
- Failure is returned to the caller
- Bounded by an integration timeout

EVENT-DRIVEN

The caller does not wait

- Work happens after the response
- A queue, topic or event bus
- Latency is absorbed by the buffer
- Failure retried, then a DLQ
- Bounded by the consumer, not the client

THE TIEBREAKER

Most scenarios contain both. The question is which parts genuinely need an answer before the user sees a response, and the answer is usually fewer than you think.

The four domains as a workflow

DOMAIN	THE QUESTION IT ASKS
1. Development, 32%	Can you build it with the right services?
2. Security, 26%	Can you make it accessible only to who should reach it?
3. Deployment, 24%	Can you ship it repeatedly and safely?
4. Troubleshooting, 18%	Can you tell what is wrong when it breaks?

ON THE EXAM

That is a developer's working week in four lines. The weights are not arbitrary: building is the largest, and troubleshooting the smallest but the most diagnostic.

Answering a which-service question

- **Find the constraint**, not the topic. The constraint is usually one clause.
- Duration, size, ordering, consistency, cost, or who must be able to do it.
- **Eliminate on hard limits first.** A stated duration or size often removes options outright.
- Then apply the qualifier: most cost-effective, least operational overhead.
- If two options still both work, you have missed a clause. Reread rather than guessing.

> ON THE EXAM
>
> Every worked scenario in this course names the deciding clause explicitly. That is deliberate, and module 30 makes the technique explicit.

What this exam does not ask

- It does not ask you to **write code**. Snippets appear, usually Python with boto3.
- It does not ask for **pricing figures**, only relative cost.
- It does not test **console click paths** or CLI flags.
- It does not go deep on **networking**. VPCs appear as constraints, not as subjects.
- It does expect **limits, defaults and which service fits**, repeatedly.

> ON THE EXAM
>
> People routinely over-prepare on networking and under-prepare on limits. The last line is where revision time actually pays.

How this course is arranged

- **Modules 2 to 14** cover Domain 1: the SDK, then the services you build with.
- **15 to 19** cover security. **20 to 24** cover deployment. **25 to 28** cover troubleshooting.
- **29 to 31** are synthesis: confusion pairs, question anatomy, and what changed since the guide.
- **32 to 34** are three practice exams, sixty questions with no repeats.
- Every teaching module ends with worked scenarios. Pause and commit before each one.

> ON THE EXAM
>
> The modules build on each other, so the order is worth following. Later modules assume the vocabulary earlier ones established.

Two scenarios. Neither needs a service you have met yet - only the ideas from this module.

A team is moving a web application to AWS. It currently stores uploaded files on the web server's local disk and keeps user sessions in the application process's memory. They plan to run it on a fleet that scales in and out with demand.

They report that after scaling, some users lose their session and some uploaded files cannot be found.

What explains both symptoms?

A The instances are stateless, so anything held in memory or on local disk is lost when one is replaced

B The load balancer is misconfigured and routing to unhealthy instances

C The instances need larger disks to hold the uploaded files

D Session data must be replicated between instances by the application

◆ PAUSE AND DECIDE

What explains both symptoms?

A Compute is stateless

Both symptoms have one cause. Scaling in terminates instances, taking their memory and local disk with them. Sessions belong in DynamoDB or ElastiCache, and files in S3.

B Load balancer misconfigured

Would cause failed requests, not selectively missing sessions and files. It also does not explain why the symptoms follow scaling events.

C Larger disks

The files are not too large, they are on an instance that no longer exists. A bigger disk on a terminated instance helps nobody.

D Replicate sessions between instances

The near miss. It addresses sessions and not files, and it builds a distributed state system rather than using a service designed for it.

THE SIGNAL

One idea explains two symptoms, which is the tell that you have found the cause rather than a cause. Option D fixes half the problem the hard way.

> An order service calls a payment provider synchronously, then an inventory service, then an email service, all within one request. When the email service is slow, order submission times out and customers see failures even though payment succeeded.
>
> Only payment confirmation must be known before responding to the customer.

What is the appropriate change?

A. Increase the timeout on the order service so slow calls complete

B. Retry the email call until it succeeds before responding

C. Respond after payment, and publish an event that inventory and email consume independently

D. Run all three calls in parallel to reduce total latency

◆ PAUSE AND DECIDE

What is the appropriate change?

A Longer timeout

Makes customers wait longer for something they do not need. The coupling that causes the failure is untouched.

B Retry email inline

Makes it worse. The customer now waits through several attempts at the slowest and least important step.

C Respond after payment, publish an event

The scenario states only payment must be known before responding. Everything else becomes an event, so a slow email cannot fail an order.

D Parallel calls

Reduces total latency and keeps the coupling: the response still waits for the slowest of the three, which is the email service.

THE SIGNAL

The deciding clause is 'only payment confirmation must be known before responding'. It tells you exactly where the synchronous boundary belongs.

What to carry forward

- **Identity is the perimeter.** When access fails, ask which identity is calling.
- **Managed means operated, not foolproof.** Most scenarios are misconfiguration.
- **Compute is stateless and everything fails**, so idempotency is a requirement.
- Same account is OR, cross-account is AND. Limits are per account and Region.
- When two options both work, prefer the more loosely coupled one.

> ON THE EXAM
>
> Next: the AWS SDK and the API request lifecycle, which is how every one of these ideas reaches actual code.